

doit

An Interactive LLM Shell Agent

Assignment 3 — Agents

LLM Course

Ido Raviel · 322758285

Ofek Chincholiker · 323811925

June 2026

a natural-language → shell agent, built incrementally across all assignment sections

Quick Reference

This table maps each part of the assignment to where it is covered in this report, the capability delivered, and the code that implements it. The agent is a single Python program (`doit`) plus modules under `src/`; runtime state lives in the hidden `~/.doit/` directory and the model is selected in `~/doit.cfg`.

§	Assignment part	What was delivered	Key code
1	Overview & architecture	LPU-system model, structured-JSON contract, the agent loop	<code>doit</code>
2	Single command at a time	NL→bash, three response types, capture stdout/stderr/rc	<code>llm.py</code> , <code>shell.py</code>
3	Identifying dangerous commands	Dual safety gate (LLM flag + regex), y confirmation	<code>safety.py</code>
4	Model flexibility	Hosted + 2 local models, one-line config switch	<code>config.py</code>
5	ACDL documentation	Context schemes + diagrams + prompt templates	<code>acdl/</code>
6	Multi-turn	Persistent history, reference resolution, no classifier	<code>history.py</code>
7	Clarifications	<code>clarify</code> type, 120 s timed wait, never guesses	<code>doit</code> , <code>llm.py</code>
8	Richer interactions	"how do I..." answers; execute / modify follow-ups	<code>llm.py</code>
9	Memory	Durable facts via a dedicated extraction call	<code>memory.py</code>
10	User awareness	<code>shell_history</code> tool + CWD injection	<code>tools.py</code>
11	Output awareness	Reasoning over stored command output/errors	<code>history.py</code>
12	Multi-tasking	Per-terminal sessions + cross-window tools	<code>history.py</code> , <code>tools.py</code>
13	Further extensions	3 described; project profiles implemented	<code>project.py</code>
14	Model comparison	mistral (tool) vs llama3 (no-tool), with failures	—
A	Prompts, tools, schemas	Full system prompt, tool registry, JSON schema	<code>appendix</code>

Models used throughout. Hosted: gemini/gemini-2.5-flash (Google AI Studio, free tier). **Local, tool-calling:** ollama/mistral:7b. **Local, no tool-calling:** ollama/llama3:8b (run with Ollama on an RTX 4070).

TODO (gemini coverage): the hosted free tier repeatedly returned HTTP 503/429 during testing, so several later sections were tested on the two local models only. Each affected example is marked with a **gemini TODO** box to be filled in once the quota allows. Sections 2–6 already include gemini output.

1. Overview & Architecture

`doit` takes a natural-language request as a command-line argument, decides what to do with it, and — when the request is a shell task — prints the command, runs it, and shows the output. It grows in capability across the assignment from a stateless translator into a session-aware agent with memory, tools, and multi-terminal support.

1.1 The LPU-system view

Following the course model, the language model is a *Language Processing Unit* (LPU): it maps an input buffer (instructions + data + tool descriptions) to an output buffer (text and structured instructions). `doit` is the surrounding **system**: it formats the input, calls the LPU, interprets the structured response, and loops. Every `doit` invocation is one user turn, but a single turn may issue several LPU calls (for tools or clarifications) before acting.

1.2 Central design decision: structured JSON, not native tool-calling

The LPU always replies with a single JSON object whose `type` field selects how the system handles it. We deliberately do **not** use a provider's native tool-calling API. The assignment requires three model classes including a *local model without tool-calling support*; native tool-calls would not work there and would split the code into per-provider paths. A plain-text JSON contract works identically on all three models, gives us full control of parsing and fallback, and is transparent to document in ACDL. The course slides frame the LPU response as simply "something the agent can interpret (JSON, code, XML, ...)", which is exactly this approach.

1.3 Components and state

Module	Responsibility
<code>doit</code>	Entry point: build prompt, run the agent loop, dispatch the response, confirm dangerous commands.
<code>src/config.py</code>	Read <code>~/doit.cfg</code> (model + API key) and set the provider key.
<code>src/llm.py</code>	System prompt, the LPU call (via LiteLLM), JSON parsing, the memory-extraction call.
<code>src/shell.py</code>	Execute a command with <code>subprocess</code> , capturing stdout, stderr, return code.
<code>src/safety.py</code>	Regex backstop that flags destructive commands.
<code>src/history.py</code>	Per-session multi-turn history; replay; pruning; cross-session access.
<code>src/memory.py</code>	Durable user memories (global, persistent).
<code>src/tools.py</code>	Tool registry (shell history, session tools) advertised to the model.
<code>src/project.py</code>	Project profiles (<code>.doit.md</code>) — the implemented extension.

Persistent state is consolidated in the hidden directory `~/.doit/` (`history.jsonl`, `memory.jsonl`), keeping the home directory clean; the only visible file is the assignment-mandated `~/doit.cfg`.

2. Single Command at a Time

2.1 What was implemented

The base agent translates a natural-language request into a single bash command, prints it, executes it, and shows the output. The model returns one JSON object whose `type` is one of three values, so the program can recognise which case it received:

- `command` — the request is a shell task → show + run the command.
- `answer` — a question or conversation → print a text reply.
- `impossible` — not achievable in the shell → explain why.

```
$ doit "list the files in my Documents folder"
# Lists files in ~/Documents
ls ~/Documents

doc1.pdf  doc2.pdf  notes.md
```

2.2 Design decisions & assignment questions

Q: How would you capture the model's command output (convenient format)?

We ask the model for one JSON object (no markdown), parse it, and read the `command` field. Execution uses `subprocess.run` with `capture_output=True`, `text=True`, and a timeout, returning `stdout`, `stderr`, and `returncode` — exactly the three signals the assignment asks for. Because many commands (`touch`, `rm`, `mkdir`) are silent on success, the program prints `✓ done` on a silent success and `x exited with code N` on failure, so the user always gets feedback.

Q: What if the request is not possible in the shell?

The model returns `type: "impossible"` with a short explanation, e.g. "order a pizza" → "I'm a shell assistant, not a food-delivery service." Nothing is executed.

Q: What about "tell me a joke" or "what can you do"?

These map to `type: "answer"` and get a friendly text reply rather than a command.

How the type is decided. There is no separate classifier call — the single LPU response carries the `type`, and a bad/!JSON reply falls back to a plain answer. One call, one decision, easy to document.

2.3 ACDL context & prompt

On a single, stateless turn the context is just the system instructions plus the user request.

```
SingleCommand[@T]: {
  S: INSTRUCTIONS
  U: env.user_request[@T]
}
```

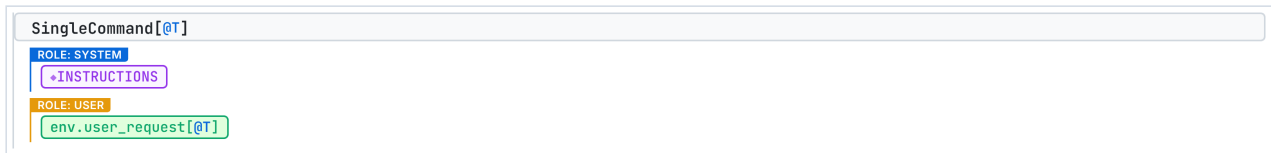


Figure 2.1 — Single-command context (rendered ACDL).

The `INSTRUCTIONS` template is the system prompt that defines the JSON contract; it is listed in full in Appendix A.

Limitations. A single stateless turn: no awareness of previous requests, the filesystem, or the user — all added in later sections.

3. Identifying Dangerous Commands

3.1 What was implemented

Read-only commands (e.g. `ls`, `grep`) run directly. Commands that modify the system (create, move, delete, overwrite, change permissions, rewrite git state) are shown with an explanation and require the user to type `y` before they run.

```
$ doit "create a file named bla.txt"
# Creates an empty file named 'bla.txt'
touch bla.txt

    This may be dangerous: creates/updates files.
Proceed? [y/N] y
✓ done
```

3.2 Design decisions & assignment questions

Q: How will you identify these cases? (one option is a separate LLM call.)

We use a **dual safety gate** (defence in depth). A command auto-runs only if *both* conditions hold:

1. the LPU tagged it `"safe": true` in the JSON, **and**
2. a local regex backstop in `safety.py` finds no destructive pattern.

Otherwise the program explains the command and waits for `y`. We do not trust the model alone for a safety-critical decision: the deterministic regex check (covering `rm`, `mv`, `dd`, `chmod`, redirections, `find ... -delete`, git history rewrites, ...) is a second, auditable opinion. The confirmation prompt is hardened against a closed input stream (EOF/interrupt are treated as "no").

ACDL. The context is unchanged from §2; only the `command` schema gains a `safe` field. The auto-run-vs-confirm decision is *program* logic, kept out of the prompt so the safety behaviour stays deterministic. (Figure 2.1 applies.)

Limitations. The regex backstop is heuristic — it can over-flag an exotic but harmless command, and it cannot understand intent. The `y` gate is the real safety net. We also observed `mistral` occasionally over-flagging a read-only pipe as unsafe via its own `safe:false` — harmless (caught by the gate), but noisier than the hosted model.

4. Model Flexibility

4.1 What was implemented

The agent runs against three model classes, selected by a single line in `~/doit.cfg`:

Class	Model	Notes
Hosted API	<code>gemini/gemini-2.5-flash</code>	Google AI Studio free tier (2.0-flash had a <code>limit:0</code> quota issue on a fresh project).
Local, tool-calling	<code>ollama/mistral:7b</code>	Served by Ollama.
Local, no tool-calling	<code>ollama/llama3:8b</code>	Served by Ollama.

There is **no model-specific code**. LiteLLM is the only thing that talks to a model and routes on the `provider/` prefix of the config string: `gemini/*` goes to Google's hosted API (key from `~/doit.cfg`); `ollama/*` goes to the local Ollama daemon at `localhost:11434`. Switching models is a one-line edit, read fresh on every invocation — no restart, no code change. This is exactly why the assignment mandates LiteLLM, and it is what makes the same structured-JSON contract portable across all three.

The detailed behavioural differences between the models are collected in §14 (Model Comparison), as the assignment requests.

5. ACDL Documentation

ACDL (Agentic Context Description Language) describes only the *context sent to the LPU* — roles, templates, data variables, and control flow over turns — not the program logic. We maintained ACDL incrementally; each context scheme has a textual block and a rendered diagram. The diagrams appear inline in the relevant sections; this section explains the notation and lists the schemes.

5.1 Notation used

- Roles: `S`: system, `U`: user, `A`: assistant.
- Namespaces: `env.*` external inputs (the user request), `sys.*` agent/environment state (cwd, memories, tools, tool results), `resp.*` prior LPU outputs.
- Templates are ALL_CAPS (e.g. `INSTRUCTIONS`, `AVAILABLE_TOOLS`); time is 1-indexed (`@T` current, `@t` a loop variable).
- Control flow: `ForEach` over a range, `If/ElseIf` conditionals. ACDL has no `While` loop, so a bounded intra-turn agent loop is modelled as `ForEach(@k: range(1, MAX_STEPS))` with a two-dimensional index `[@T, @k]` (the k -th LPU call within turn T).

5.2 Context schemes across the assignment

Scheme	Introduced in	What changed
Base (stateless)	\$2 single command	<code>S</code> : <code>INSTRUCTIONS</code> , <code>U</code> : request
(prompt-only)	\$3 dangerous	adds <code>safe</code> field; same scheme
+ history	\$6 multi-turn	replay prior turns
+ clarify loop	\$7 clarifications	intra-turn <code>If clarify</code>
+ memory	\$9 memory	<code>sys.memories</code> injected; separate extractor context
+ tools, cwd	\$10 user awareness	<code>AVAILABLE_TOOLS</code> , <code>sys.cwd</code> , tool loop
+ sessions	\$12 multi-tasking	session-scoped history; multi-step <code>ForEach(@k)</code>
+ project	\$13 extension	<code>sys.project</code> injected

The full prompt templates for each are in Appendix A.

6. Multi-turn

6.1 What was implemented

Because each `doit` run is a separate process, history is persisted so follow-ups can refer to earlier turns. After acting, the program appends one record to `~/.doit/history.jsonl`; on the next run it replays the recent turns into the context.

```
$ doit "list the files in this directory"
ls                               → acdl doit report src ...
$ doit "now sort them by size, largest first"
ls -lS                          → (sorted listing)
$ doit "no, smallest first"
ls -lS | tail -n +2 | sort -h
```

Turns 2 and 3 resolve "them" and the correction against the earlier turns — across three separate processes.

6.2 Design decisions & assignment questions

Q: How and where do you store the history?

Append-only JSON Lines at `~/.doit/history.jsonl`. Each record stores the timestamp, session id, the request, the type, the command + explanation, and the execution result (stdout/stderr/return code, plus an `executed` flag). JSONL suits a fresh process per turn — append one line, tail the last N — with no rewrite.

Q: How do you present the history to the LLM?

We replay the last `MAX_TURNS = 8` turns as real user/assistant message pairs. The user message is the old request; the assistant message is a rendered view (for a command: `Ran: <cmd>` + a clipped output tail + exit code; for an answer: the text). Clipping bounds the context; the window bounds drift.

Q: How do you distinguish new commands from ones that refer to previous turns — and which one?

Deliberately **no separate classifier**. We feed the bounded recent history and let the model decide whether a request is fresh or a reference, and which prior turn it points at. A hard "is-this-a-follow-up?" gate is brittle on indirect references; replaying real turns is simpler and degrades gracefully.

```
MultiTurn[@T]: {
  S: INSTRUCTIONS
  ForEach(@t: range(1, @T-1)) {
    U: env.user_request[@t]
    A: resp.replay[@t]
  }
  U: env.user_request[@T]
}
```

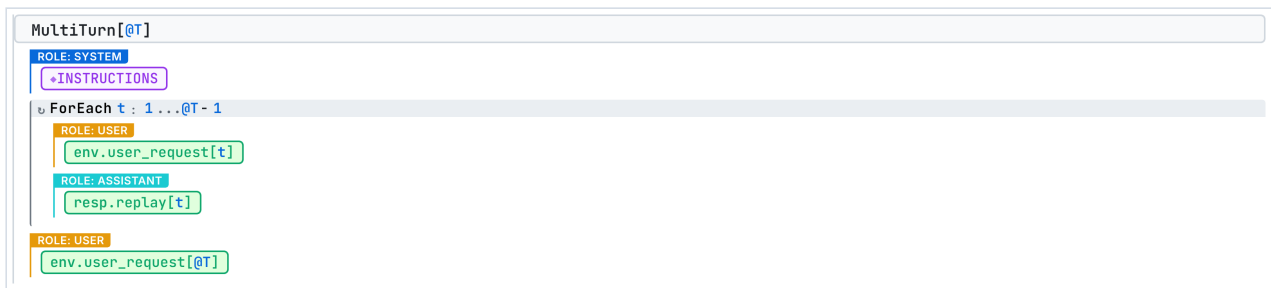


Figure 6.1 — Multi-turn context: system + replayed history + current request.

Limitations. Only the last 8 turns are replayed (older context is dropped — addressed conceptually by the "context compaction" extension in §13). Each record is also tagged with the producing `model` for traceability only — it is not fed back into the context.

7. Clarifications

7.1 What was implemented

A fourth response type, `clarify`, lets the model ask a question with options instead of guessing. `doit` prints them, waits for an answer, then re-asks the model with the answer appended.

```
$ doit "sort the files here by date"
Which date do you want to sort by?
1. modification date
2. access date
3. creation date
Your answer: 2
# ...sorted by access date...
ls -tlu
```

7.2 Design decisions & assignment questions

Q: How do you identify uncertainty (useful but not annoying)?

The prompt instructs the model to prefer a sensible default and use `clarify` only when the options lead to *materially different* commands with no clearly-best default (e.g. creation vs. access vs. modification date). So it asks rarely.

Q: What if the user does not answer?

The agent **never guesses**. It waits up to `CLARIFY_TIMEOUT = 120 s` (a `select`-based timed read on `stdin`) and then exits cleanly with a message, recording an unanswered turn. The loop is bounded by `MAX_CLARIFY = 3` rounds.

Q: How is the mechanism implemented (the question as a tool)?

As a structured-JSON response type rather than a native tool — the same portability reason as in §1.2. So `command` / `answer` / `impossible` / `clarify` are four shapes of one response, not four tools. ACDL has no `While`, so the round-trip is modelled with an `If resp.type == clarify` block (the bound lives in code).

```
Clarify[@T]: {
  S: INSTRUCTIONS
  ForEach(@t: range(1, @T-1)) { U: env.user_request[@t]  A: resp.replay[@t] }
  U: env.user_request[@T]
  If resp.type[@T] == clarify {
    A: resp.clarify[@T]
    U: env.clarify_answer[@T]
  }
}
```

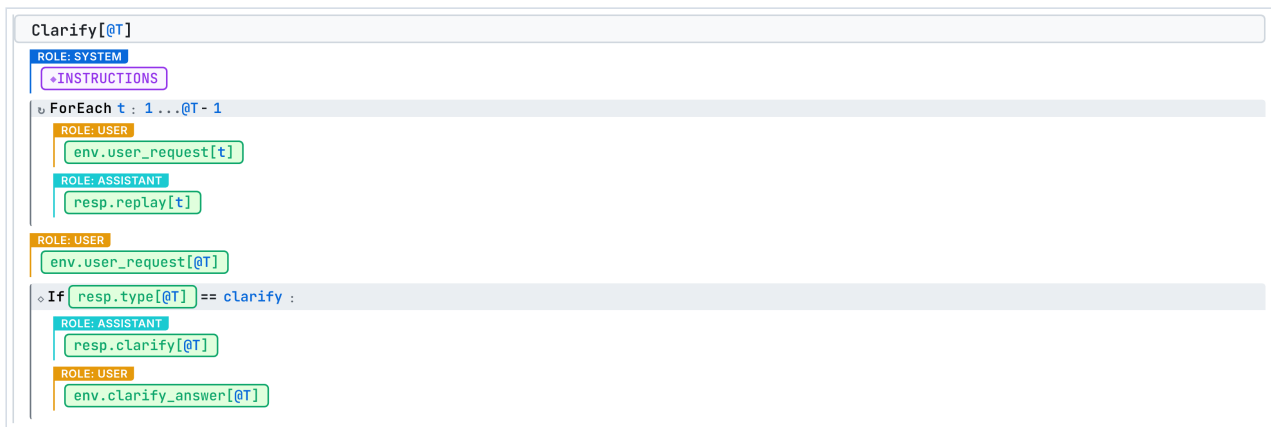


Figure 7.1 — Clarification adds a conditional intra-turn exchange.

Limitations. Whether to ask is the model's judgement and is model-dependent (see §14): on the same prompt the hosted model asked while the local models defaulted.

8. Richer Interactions

8.1 What was implemented

The agent answers non-command questions ("how do I...") with an explanation and a *suggested* command (not executed), and supports follow-ups: "execute it" runs the suggestion, "modify it to..." updates it.

```
$ doit "how do I list the 3 largest files here?"
You can list them with `ls -lS | head -n 3`. ...      (answer – nothing run)
$ doit "modify it to show the 5 largest"
Use `ls -lS | head -n 5`.                             (still a suggestion)
$ doit "execute it"
ls -lS | head -n 5                                    (now it runs)
```

8.2 Design decisions & assignment questions

Q: How to make it work? What happens in longer sequences?

This is a prompt-only change on top of multi-turn: the prompt distinguishes asking *how* (an `answer` with the command in backticks) from asking to *do* (a `command`), and resolves "execute it" / "modify it" against the replayed history (an executed command appears as `Ran:`; a suggestion appears as plain text). Notably, rules alone were not enough for `mistral` — it kept turning "modify it" into an execution; a single worked **example** added to the prompt fixed it, while the hosted model obeyed the rules without one. (No new ACDL scheme; Figure 7.1 applies.)

Limitations. Command *quality* on the follow-ups is model-dependent (§14): the no-tool model produced logically poor commands even when the conversational flow was right.

9. Memory

9.1 What was implemented

Durable facts/preferences about the user persist across terminals, directories, and sessions in `~/.doit/memory.jsonl`, and are injected into every prompt so the agent can act on them later. The same request can both act and store a memory.

```
$ doit "the folder ~/school/llms/ass3 is my LLM class project folder"
  Remembered: The user's LLM class project folder is ~/school/llms/ass3.
... (acknowledgement)

# later, in a brand-new terminal (history cleared, memory kept):
$ doit "what is the path to my llm class project?"
The path to your LLM class project is ~/school/llms/ass3
```

9.2 Design decisions & assignment questions

Q: How are memories stored and when?

Memory is **orthogonal to the response type** (a turn can act *and* remember), so it is not a type. Deciding what to save is done by a **dedicated extraction call** that runs every turn: a focused second LPU call looks at the request and decides "is there a durable fact? if so, write it concisely." We use a separate call because the weaker local models could not reliably add a memory field *and* answer in one response — a clear instance of the two-jobs-in-one-output problem. Storage is a plain append.

Q: How do memories appear in the LLM context?

All stored memories are rendered into the system prompt every turn (`sys.memories`), so even a brand-new terminal sees them.

Q: What about "I changed my mind ... ask me each time"?

Memories are listed oldest→newest and the model is told later entries override earlier ones, so a new preference supersedes an old one. To avoid re-saving a fact the user merely *asks about*, the extractor is shown the already-known memories ("Already known: ...") and returns `save:false` for anything already stored — this fixed a duplicate-on-recall bug.

```
MemoryAware[@T]: {
  S: { INSTRUCTIONS sys.memories }
  ForEach(@t: range(1, @T-1)) { U: env.user_request[@t] A: resp.replay[@t] }
  U: env.user_request[@T]
}
// separate extraction context:
ExtractMemory[@T]: { S: MEMORY_INSTRUCTIONS U: { env.user_request[@T] resp.action[@T]
sys.memories } }
```

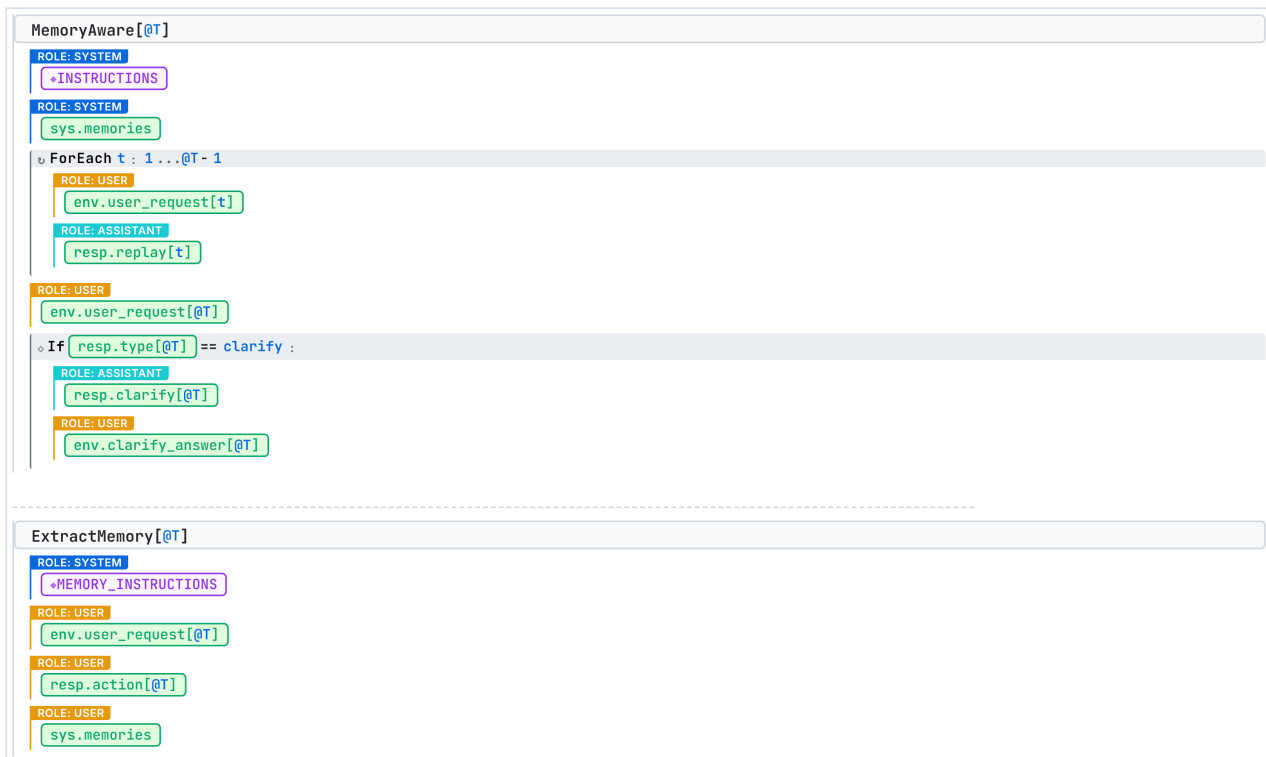


Figure 9.1 — Two contexts per turn: the main context (with memories injected) and the dedicated extraction call.

gemini TODO: on gemini, **store** was tested (the memory was saved with a clean acknowledgement); the **recall** turn could not be captured because the free tier returned 503/429. Re-run the store→recall sequence on gemini and insert the recall transcript here.

Limitations. Factual recall works well; *behavioural* preferences ("ask me each time") are shaky on mistral (it read the preference as an action and did not store it). The combined action+memory request is the hardest case — see §14. Injecting all memories into every prompt grows with memory count; a retrieval/summarisation layer (§13) would bound it. Reading the cwd is free, but a `cd` the agent runs cannot change the user's real shell directory (a subprocess limitation).

10. User Awareness

10.1 What was implemented

The agent becomes aware of the *user's* own actions and location two ways: the current working directory is injected into the prompt every turn (free — `doit` inherits the shell's directory), and a `shell_history` **tool** returns the user's recent interactive commands on demand. This is the explicit tool-call loop the course unit is about — implemented as roll-your-own tool use (a `tool` response type), so it works on all three models.

```
# user has run, in their shell: cd ~/school/llms/ass3 ; mkdir data ; python train.py ; ls -la
$ doit "summarize what I just did in the shell"
You navigated to your school project directory, created a 'data' folder, ran the
Python script 'train.py', and listed the directory contents in long format.
```

User vs. `doit` actions separate **naturally**: `doit` runs commands via `subprocess`, which never touch the interactive bash history, so `shell_history` returns only the user's manual commands (and `doit ...` invocation lines are filtered out). A one-line `PROMPT_COMMAND='history -a'` snippet in `~/.bashrc` (documented in the README) keeps history fresh in-session.

10.2 Design decisions & assignment questions

Q: How to integrate the user's behaviour, and combine it with the agent's, in the context?

CWD is always pushed into the system prompt; shell history is pulled via a tool only when relevant (e.g. "what did I just do"). The agent's own actions remain in `~/.doit/history.jsonl`; the user's are in bash history — two clearly separated streams. Tools are advertised to the model in the system prompt (`AVAILABLE_TOOLS`), mirroring the course example that lists `use_google(...)` / `use_healthline(...)`.

```
UserAware[@T]: {
  S: { INSTRUCTIONS AVAILABLE_TOOLS sys.cwd[@T] sys.memories }
  History { ForEach(@t: range(1, @T-1)) { U: env.user_request[@t] A: resp.replay[@t] } }
  U: env.user_request[@T]
  If resp.type[@T] == tool {
    A: resp.tool_call[@T]
    U: sys.tool_result[@T]
  }
}
```

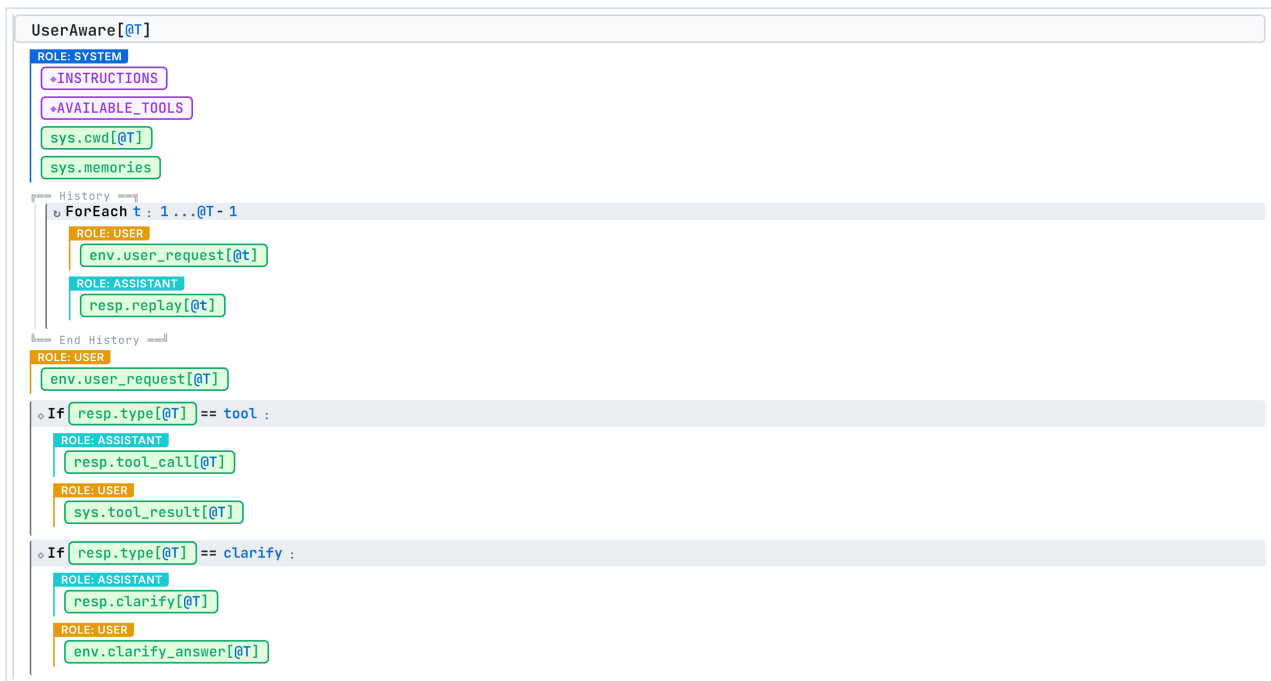


Figure 10.1 — Tools + CWD in the system block; a tool call feeds its result back.

gemini TODO: the `shell_history` tool call was tested on `mistral` and `llama3` (`mistral` used it correctly; `llama3` did not — see §14). Re-run "summarize what I just did" on `gemini` and insert the transcript here.

11. Output Awareness

11.1 What was implemented

Questions about a previous command's output or errors are answered directly, because the prior command's stdout/stderr/exit-code are already stored and replayed. No new code was needed.

```
$ doit "show the contents of /etc/sudoers"
cat /etc/sudoers          → cat: /etc/sudoers: Permission denied  (exit 1)
$ doit "why did that fail?"
It failed because you don't have read access to /etc/sudoers; try `sudo cat /etc/sudoers`.

$ doit "show the 5 largest files here" → (a listing)
$ doit "which of these looks safe to delete?"
doit.cfg.example is small and just an example config – likely safe; back it up first.
```

11.2 Design decisions & assignment questions

The replayed assistant turn (resp.replay) already carries Ran: <cmd> plus a clipped stdout/stderr tail and the exit code, so output awareness is a property of the multi-turn design rather than a new mechanism. A small prompt rule was added to ground failure explanations in the real stored output and to **not** claim a "Proposed (not run)" command failed (it correctly reports such a command as not executed). No new ACDL scheme (Figure 10.1 applies).

Push vs. pull. Output awareness is "push" — the output is already in the context, so even the no-tool model handles it well; contrast the shell-history tool in §10, which is "pull" and requires the model to decide to call it (where the no-tool model failed). This contrast recurs in §14.

12. Multi-tasking

12.1 What was implemented

Each terminal gets a `session` id, so each window keeps its own history stream, while explicit cross-window references are still possible. The session id is `$DOIT_SESSION` (set by an optional `.bashrc` snippet) or, by default, the launching shell's process id — stable across invocations in one terminal and different between terminals, so it works with no setup.

Session isolation

```
Window 1: doit "list the files in this directory"      → ls (+ listing)
Window 2: doit "create a folder for each year 2020..2026" → for i in {2020..2026}; do mkdir
-p ./years/$i; done
Window 1: doit "sort them by size"                    → ls -lS    (sorts W1's listing,
NOT W2's folders)
```

Cross-window reference

```
Window 1: doit "now do the same folder task we did in the other window, here"
          for i in {2020..2026}; do mkdir -p ./years/$i; done    (reproduced W2's exact
command)
```

With memory cleared and Window 1's own history holding no folder task, the only way to obtain that command is via the tools: the model calls `list_sessions` (a catalogue of the other windows with a short summary of each) to find Window 2 by its description, then `session_history(id)` to read its commands.

12.2 Design decisions & assignment questions

Q: How to make the agent aware of which stream it is running in, when history is several interleaved streams?

By default `load_recent()` filters history to the current session, so each window only sees its own turns. The user never knows the internal session id, so cross-window works as **two-step tool use**: `list_sessions` returns id + summary; the model matches the user's phrase ("the other window") to a session and only then calls `session_history` with that id; genuine ambiguity falls back to a clarification.

Q: What about closed terminals?

Cleanup is **lazy and staleness-based** — sessions idle for more than 7 days are pruned on the next run — rather than tied to a terminal-close hook, because close events are unreliable (`kill`, crash, SSH drop, reboot do not fire a hook).

```

MultiTask[@T]: {
  S: { INSTRUCTIONS AVAILABLE_TOOLS sys.cwd[@T] sys.memories }
  History { ForEach(@t: sys.session_turns[@T]) { U: env.user_request[@t] A:
resp.replay[@t] } } // current session only
  U: env.user_request[@T]
  ForEach(@k: range(1, MAX_STEPS)) { // intra-turn agent loop
    If resp.type[@T,@k] == tool { A: resp.tool_call[@T,@k] U: sys.tool_result[@T,@k] }
    ElseIf resp.type[@T,@k] == clarify { A: resp.clarify[@T,@k] U:
env.clarify_answer[@T,@k] }
  }
}

```

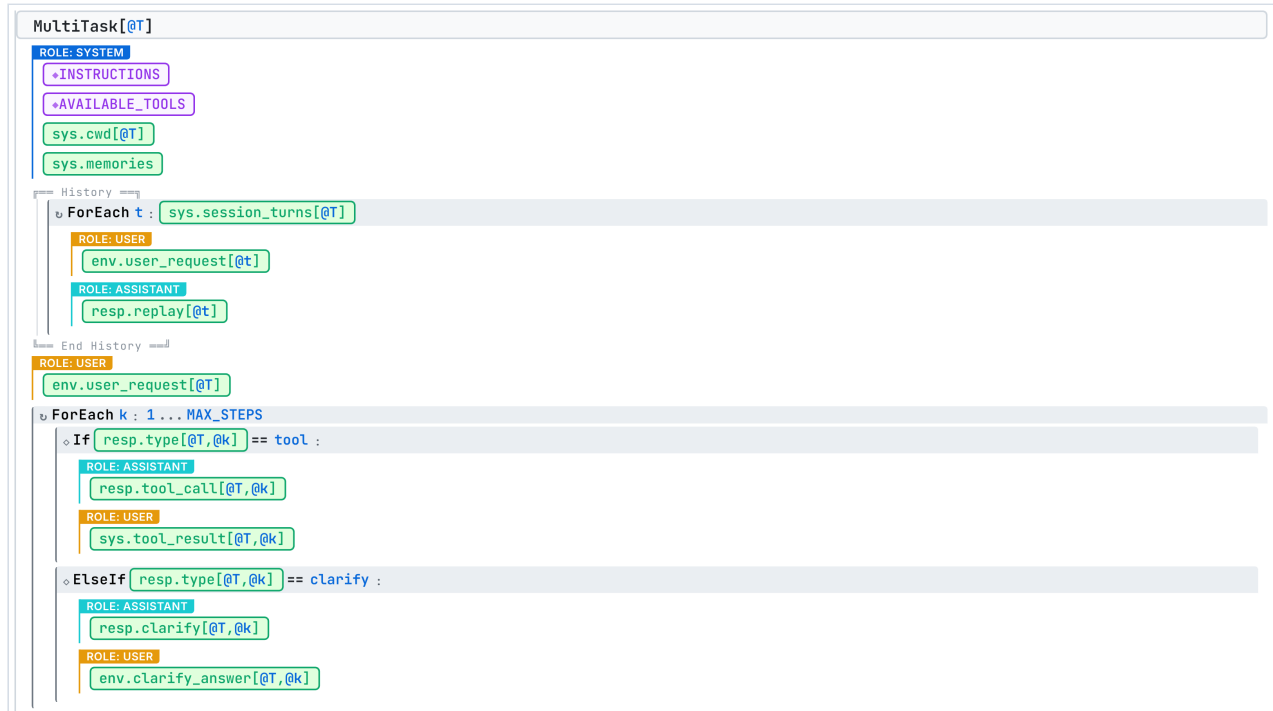


Figure 12.1 — Session-scoped history and the multi-step tool/clarify loop. The cross-window flow runs as @k=1 list_sessions → @k=2 session_history → @k=3 the command.

This multi-step `ForEach(@k)` loop mirrors the agentic loop from the course (observe → select action → act → observe response): the model emits a structured request, the environment executes it and feeds the result back, and the loop continues until the model produces a final command or answer.

gemini TODO: isolation and cross-window were tested on `mistral` (reliable) and `llama3` (inconsistent — see §14). Re-run the cross-window sequence on `gemini` and insert the transcript here.

13. Further Extensions

The assignment asks for three extensions to be described and one implemented. We note first that multi-step *tool* use already exists in the core (the `ForEach(@k)` loop chains tool/clarify steps, §12), so it is not used as the additional extension.

13.1 Three candidate extensions

Candidate	What it adds	How it would be built	Why harder
Context compaction / summarisation	Summarise turns that fall out of the 8-turn window into a running "summary so far", so long sessions keep early context.	A dedicated summarisation call over the oldest turns; store a per-session summary and prepend it; refresh as turns age out.	Extra model call (cost + quality risk: a bad summary silently corrupts context) and invalidation logic.
Project profiles (implemented)	Per-folder <code>.doit.md</code> instructions so the agent behaves differently per project (the <i>agent.md</i> idea).	Find the nearest <code>.doit.md</code> and inject it into the prompt.	Easiest — pure context injection, no new call/type/loop.
Command plans	One request → a proposed numbered sequence of shell commands, shown, then executed step-by-step.	A new plan type + an execution loop reusing the safety gate and output feedback; replan on a failed step.	New type + multi-command execution with per-step confirmation and failure handling.

13.2 Implemented: project profiles

We implemented project profiles because it is clearly additional and reuses the existing context-injection pattern with no new model call, response type, or loop, yet is a genuine capability (per-project behaviour / context management). `doit` finds the **nearest** `.doit.md` — starting at the current directory and walking *up* through parents, using the first one found (like `git` locating `.git`) — caps it, and injects it as a "Project profile" block. A profile thus applies anywhere below it; a closer profile in a subfolder overrides a more general one higher up; if none exists, nothing is injected.

```
# /tmp/projtest/.doit.md :
# - When listing files, ALWAYS use long format with human-readable sizes (ls -lh).
$ cd /tmp/projtest      && doit "list the files here"    → ls -lh
$ cd /tmp/projtest/sub  && doit "list the files here"    → ls -lh  (walk-up found the
profile)
```

```

ProjectProfile[@T]: {
  S: { INSTRUCTIONS AVAILABLE_TOOLS sys.cwd[@T] sys.project[@T] sys.memories }
  History { ForEach(@t: sys.session_turns[@T]) { U: env.user_request[@t] A:
resp.replay[@t] } }
  U: env.user_request[@T]
  ForEach(@k: range(1, MAX_STEPS)) { If resp.type[@T,@k] == tool { ... } ElseIf ... clarify
{ ... } }
}

```

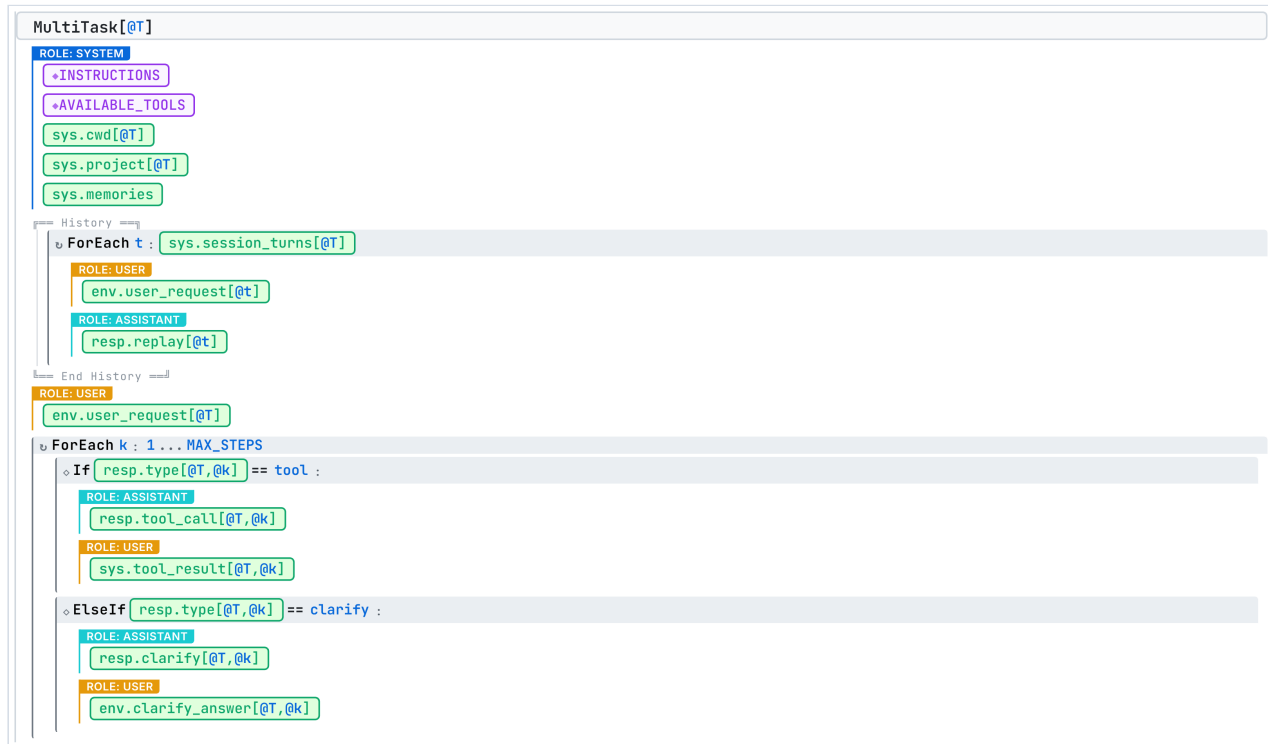


Figure 13.1 — The extension adds one line, `sys.project[@T]`, to the system block.

Because the profile is pushed into context (not a tool to decide to call), **both** local models honoured it (each produced `ls -lh`), including from a nested subdirectory.

gemini TODO: project profiles tested on the two local models; re-run on gemini and confirm `.doit.md` is honoured.

14. Model Comparison

The assignment asks for a comparison between two local models — one trained for tool calling (`mistral:7b`) and one not (`llama3:8b`) — including at least one interaction where the weaker model failed or needed different prompting. The hosted model (`gemini-2.5-flash`) is included where output was captured before the free-tier limits.

14.1 Summary across capabilities

Capability	gemini (hosted)	mistral:7b (tool)	llama3:8b (no-tool)
Structured JSON contract	clean	clean	clean on easy cases
Multi-turn references	✓ simplest commands	✓ correct	✓ resolves, but poor commands
Clarify when ambiguous	✓ asked	defaulted (didn't ask)	defaulted + broken command
"modify it" stays a suggestion	✓ from rules	only after an example	flow ok, command poor
Memory: combined action+memory	TODO	saved memory, did <i>not</i> run	ran command, did <i>not</i> save
Shell-history TOOL (pull)	TODO	✓ called it correctly	✗ never called it; hallucinated
Output awareness (push)	TODO	✓	✓
Cross-window TOOL (pull)	TODO	✓ reliable	inconsistent (worked once, then failed)
Project profile (push)	TODO	✓	✓

14.2 The central finding

The non-tool model handles the *structure* (valid JSON, correct response type, reference resolution) but is markedly weaker at the two things the tool-trained model does well: producing **correct shell commands** and **deciding to call a tool**. A clear divide also emerges between **push** capabilities (information already in context — output awareness, project profiles) which even `llama3` handles, and **pull** capabilities (a tool the model must choose to call — shell history, cross-window) which `llama3` performs unreliably.

14.3 Concrete failure cases

(a) Command logic — sort direction inverted and output garbled (llama3)

On "sort the files by size, largest first" then "smallest first":


```

gemini : ls -S          then ls -Sr          (correct)
mistral: ls -lS         then ls -lS | tail -n +2 | sort -h (correct)
llama3 : ls -lSr | grep ^- | cut -d' ' -f9- (WRONG)
        → -r reverses, so "largest first" became smallest first;
        → cut -d' ' splits on a single space but ls -l pads with runs of
          spaces, so the output was garbled (truncated/split names).

```

(b) Needed different prompting — "modify it" (mistral)

After a "how do I..." answer, "modify it to show 5" should stay a *suggestion*. `gemini` obeyed the rule directly; `mistral` kept turning it into an executed command until a single worked **example** was added to the prompt, after which it complied. This shows the weaker instruction model needs examples, not just rules, for subtle behaviour.

(c) Tool-call decision — shell history (llama3)

```

# seeded shell history: cd /opt/zephyr-quux ; ./run_glorbax.sh --froblicate 7 ; export
WIBBLE=42
$ doit "what did I just run in my shell?"
mistral: "You recently ran cd /opt/zephyr-quux, ./run_glorbax.sh --froblicate 7,
          and export WIBBLE=42."          (called the tool; recited the real commands)
llama3 : "You ran `Ran: ls -l`."          (HALLUCINATED – never emitted a tool request;
                                          even echoed the internal 'Ran:' replay format)

```

(d) Combined action + memory — each local model gets one half

```

$ doit "list all the files in ~/school/llms/ass3, this is my llm class project folder"
mistral: saved the memory, but only SUGGESTED the listing (did not run it)
llama3 : RAN the listing, but did NOT save the memory
          (llama3's extractor returns {"save": false} on this input, so nothing is stored)

```

Because the action and the memory decision are two independent LPU calls, one can succeed while the other fails — and the two weak models fail on opposite halves. The hosted model is the one we would expect to do both; that run is a `gemini TODO`.

gemini TODO (this section): fill in the gemini column for memory, the shell-history tool, output awareness, cross-window, and project profiles once the free-tier quota allows the runs to complete.

Appendix A — Prompts, Tools & Schemas

A.1 Response schema (the JSON contract)

```
command : {"type":"command", "command":"<bash>", "explanation":"<one line>", "safe":<true|false>}
answer  : {"type":"answer", "text":"<reply>"}
impossible:{"type":"impossible", "text":"<why>"}
clarify  : {"type":"clarify", "question":"<q>", "options":["<opt>", ...]}
tool     : {"type":"tool", "tool":"<name>", "args":{" ... }}
```

A.2 System prompt (INSTRUCTIONS), abridged

You are a shell assistant. Respond with a single JSON object – no markdown.
 The "type" field is exactly one of: command, answer, impossible, clarify, tool.

1. command – a shell task. Set "safe" false if it creates/moves/deletes/overwrites files, changes permissions/ownership, formats disks, or rewrites git state.
2. answer – a question/conversation, or a "how do I..." request: explain and put the suggested command in backticks, but do NOT execute it.
3. impossible – cannot be done in the shell.
4. clarify – genuinely ambiguous AND the choice materially changes the command.
5. tool – you need info only a tool can provide; available tools are listed below.

Rules (excerpt): prefer a sensible default over asking; "how do I" = answer, imperative = command; "execute it/run it" runs the previous suggestion; "modify it to..." keeps the same mode. To explain a failure use the earlier turn's real output + exit code; a "Proposed (not run)" command was not executed. Use stored memories to resolve references. [+ a worked example of a how-do-I thread]

A.3 Memory-extraction prompt (MEMORY_INSTRUCTIONS), abridged

Decide whether a candidate fact is worth saving to LONG-TERM memory (durable facts/preferences across sessions). Do NOT save transient task details, command output, activity summaries, or one-off actions ("delete the .log files" is an action, not a preference). Save only NEW info the user asserts; if it is already in the "Already known" list, return save:false. Reply: {"save":<bool>, "text":"<concise fact>"}

A.4 Tool registry (advertised as AVAILABLE_TOOLS)

Tool	Purpose	Args
shell_history	The user's recent interactive shell commands (not doit's own).	{"n": <count>}
list_sessions	Other terminal sessions, each with a summary (for cross-window references).	none
session_history	Recent turns of another session, to reuse what was done there.	{"session": "<id>"}

A.5 Safety backstop (regex patterns, excerpt)

```
rm, rmdir, mv, dd, mkfs, chmod, chown, truncate, shred, mkdir, touch, ln,  
>/>> (redirections), find ... -delete, find ... -exec, git reset|clean|checkout|rebase
```

Appendix B — Interaction Logs

These are captured runs from development. The same fixed sequence was run against each model, with `~/doit/` state reset between models so streams did not leak. They include successful cases, failure cases, and a recovery case, and they are the basis for the comparison in §14.

B.1 Multi-turn & model flexibility — the same 4-turn sequence on all three models

Sequence: (1) "list the files in this directory", (2) "now sort them by size, largest first", (3) "no, smallest first", (4) "what can you do?".

```
gemini-2.5-flash
T1  ls
T2  ls -S
T3  ls -Sr
T4  (answer) I can help you with shell commands! Tell me what you want to do, and I'll
    provide the command, an explanation, and tell you if it's safe to run...

mistral:7b
T1  ls
T2  ls -lS
T3  ls -lS | tail -n +2 | sort -h
T4  (answer) I can execute shell commands, manage files and directories, navigate the
    file system, interact with processes, and provide information...

llama3:8b
T1  ls
T2  ls -lSr | grep ^- | cut -d' ' -f9-      ← "largest first" but -r = smallest first;
                                           cut -d' ' garbles ls -l's padded columns
T3  ls -lSr | grep ^- | cut -d' ' -f9- | tac
T4  (answer) I can run shell commands, answer questions, and provide information about
    previous turns...
```

All three produced valid JSON, the right response type each turn, and resolved the follow-up references; the divergence was command quality (llama3 inverted the sort and garbled the output).

B.2 Tool use — "what did I just run?" (the clearest model divergence)

Bash history seeded with three deliberately unguessable commands so the answer can only come from actually calling the `shell_history` tool.

```
# seeded: cd /opt/zephyr-quux ; ./run_glorbax.sh --frobnicate 7 ; export WIBBLE=42
$ doit "what did I just run in my shell?"

mistral:7b → "You recently ran `cd /opt/zephyr-quux`, `./run_glorbax.sh --frobnicate 7`,
              and `export WIBBLE=42`."                (called the tool; recited the real commands)

llama3:8b → "You ran `Ran: ls -l`."                    (FAILURE: never emitted a tool request;
                                                         hallucinated, echoing the internal 'Ran:'
format)
```

B.3 Output awareness — explaining a real failure, and a recovery case

```
$ doit "show the contents of /etc/sudoers"
cat /etc/sudoers
cat: /etc/sudoers: Permission denied          (exit code 1)
$ doit "why did that fail?"
mistral: "...failed because you don't have read access to /etc/sudoers; try `sudo cat /etc/sudoers`."
llama3 : "...the cat command failed for lack of permissions; exit code 1 = permission error."

# recovery case — an aborted (never-run) command must not be reported as a failure:
$ doit "delete everything in this directory"    → rm -rf *   (dangerous; aborted at the
prompt)
$ doit "why did that fail?"
"The proposed command would delete all files recursively ... I won't execute it without
confirmation ... type 'execute it' ..."      (correctly: not run, not a failure)
```

B.4 Multi-tasking — session isolation and a cross-window reference

```
Window 1: doit "list the files in this directory"      → ls
Window 2: doit "create a folder for each year 2020..2026" → for i in {2020..2026}; do mkdir
-p ./years/$i; done
Window 1: doit "sort them by size"                    → ls -lS   (sorts W1's listing,
not W2's folders)

Window 1: doit "now do the same folder task we did in the other window, here"
mistral:7b → for i in {2020..2026}; do mkdir -p ./years/$i; done   (reliable: used
list_sessions + session_history)
llama3:8b → reproduced it once, but on a re-run ignored the reference and just did `ls`
(INCONSISTENT)
```

gemini TODO: the logs above for memory recall, the shell-history tool, multi-tasking, and project profiles were captured on the two local models only (the hosted free tier hit 503/429). Re-run those sequences on gemini and append the transcripts here.